

```
class Employee(models.Model):
    name=models.CharField(max_length=25)

class Division(models.Model):
    div_name=models.CharField(max_length=25)
```

Command Line - INPUT

```
(d_env) project_d % | python manage.py makemigrations
employee_learning
```

You can see that the migration file *0002_division.py* is created under the migrations directory.

Command Line - RESPONSE

```
Migrations for 'employee_learning':
employee_learning/migrations/0002_division.py
- Create model Division
```

The benefit of having two commands (*makemigrations* and *migrate*) in the database migration is that you can carefully manage the process. The *makemigrations* step is especially helpful when you are updating the model after the database was populated with data.

For example, when new data fields are added to the *models.py* file, the command initiates the interactive mode and asks how to handle the data for new fields.

Note: App name for the makemigrations command

You can run the *makemigrations* command without specifying an app name to create a migration file, but it is recommended to specify an app name to avoid potential future troubles.

For example, when you are making two apps in the same project, you may have two *models.py* files in each of the two apps: one with a completed model design and the other still in process of being designed. In that case, if you run the *makemigrations* command without specifying an app name, Django will create migrations files for both apps. A migration file created by the